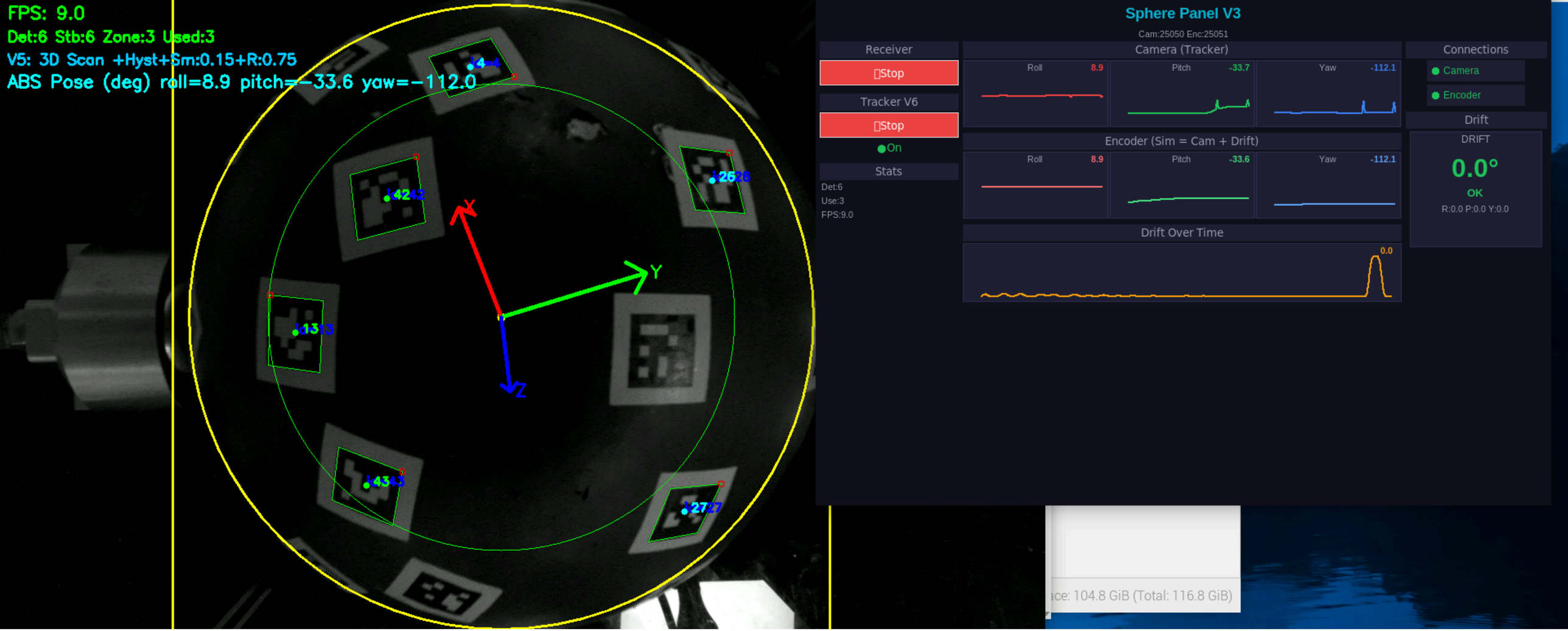
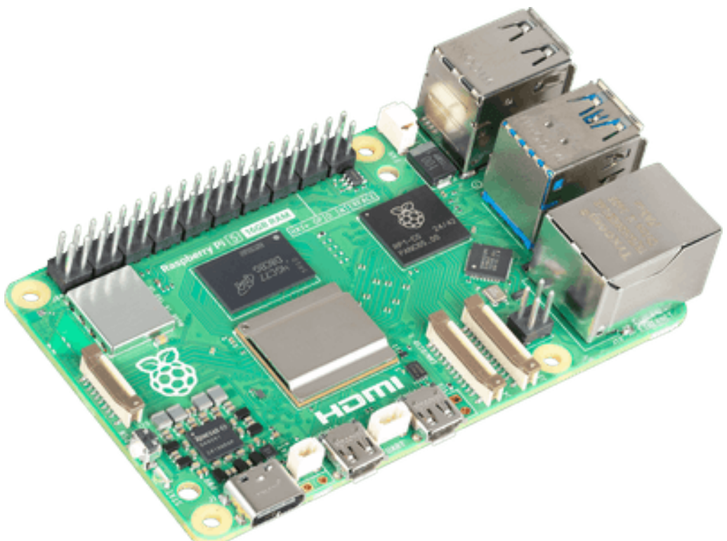


Vision-Based Orientation Tracking

Mathematical & Algorithmic Approach

Hardware Setup



Rasperry Pi 5 - 8GB



Arducam OV9281 1MP Monochrome Global Shutter Camera Module





Global Shutter



Rolling Shutter





Motion Blur



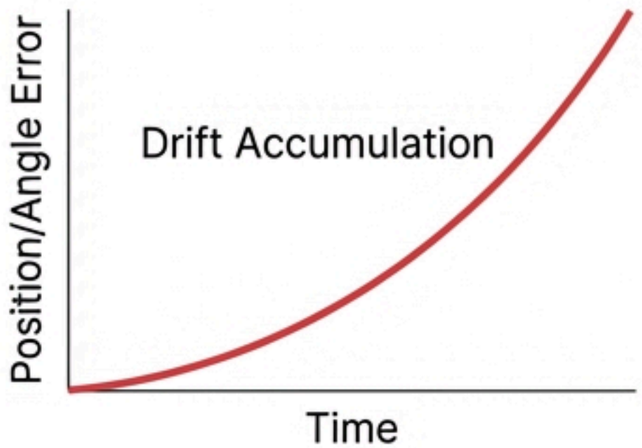
Rolling Shutter



Global Shutter

The Mathematical Problem: Dead-Reckoning vs. Geometric Truth

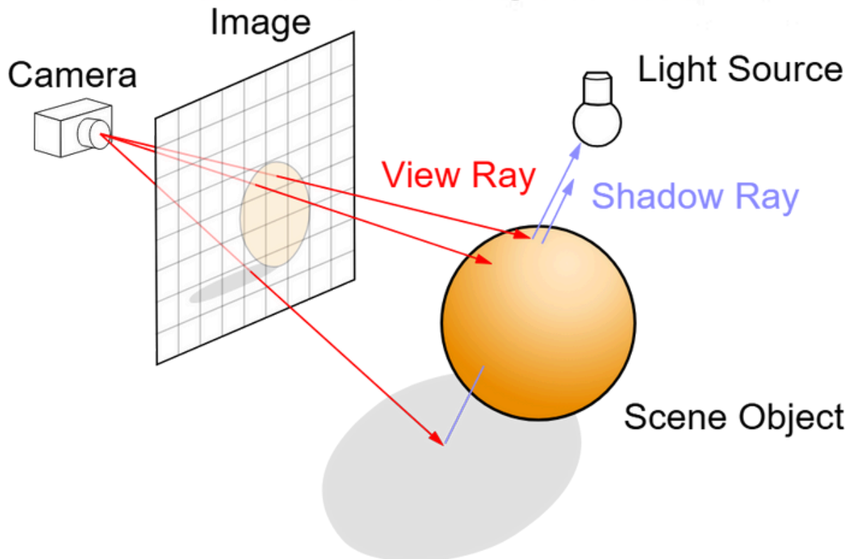
Encoder Integration



$$\theta(t) = \theta_{est}(t) - \int \varepsilon(\tau)\, d\tau$$

Encoders rely on integrating velocity over time. Slip errors (ε) accumulate indefinitely. This is infinite drift.

Vision Snapshot

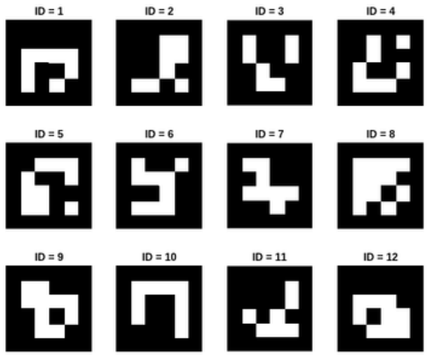


$$R_{abs} = f(Geometry_{now})$$

Vision solves a geometric snapshot at every frame. Zero drift accumulation. Errors are local, not integral.

Encoders guess where we are based on where we've been.
Vision calculates where we are based on what we see.

The Algorithm Pipeline



Input: Raw Image

↓

Step 1: Detect ArUco Markers (2D Pixel)

↓

Step 2: Undistort & Project (Pixel → 3D Ray)

↓

Step 3: Ray-Sphere Intersection (Geometric Solver)

↓

Step 4: Compute Observed Unit Vectors (v_cam)

↓

Step 5: Match with Known Vectors (v_sphere)

↓

Step 6: Kabsch Algorithm (SVD Rotation Est.)

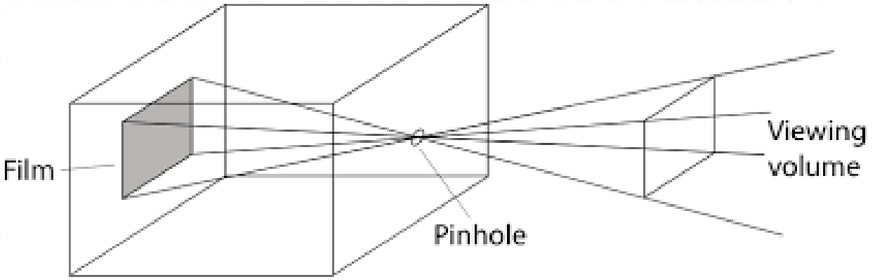
↓

Output: Euler Angles (Roll, Pitch, Yaw)

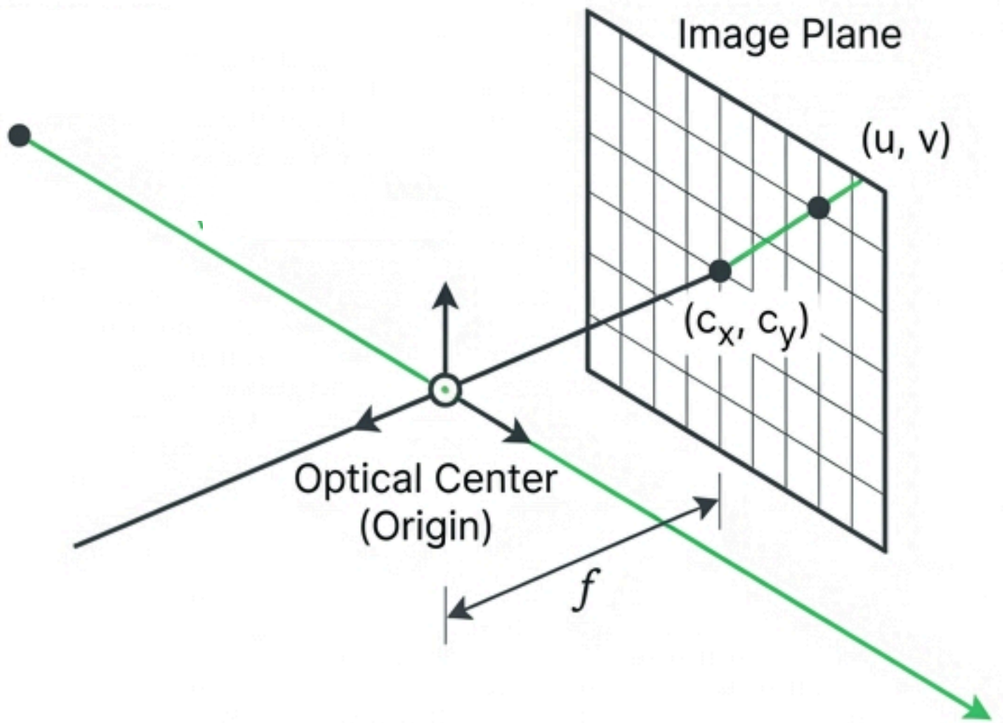
Geometric Transformation Kernel

Step 1: Modeling the Camera

Mapping the 3D world to the 2D sensor plane.



Left Side (The Model)



Pinhole Camera Model

Intrinsic Matrix (K)

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

f_x, f_y: Focal lengths (pixels)

c_x, c_y: Principal point / Optical center

Distortion Model (Brown-Conrady)

$$x_d = x_u(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + \dots$$

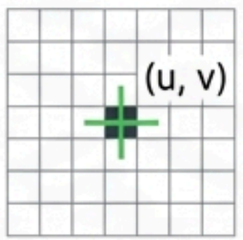
Calibrated using Zhang's method with 8×5 chessboard.

Step 2: From 2D Pixel to 3D Ray

Back-projecting the sensor data into 3D space.

Detection

Input Marker Center (u, v)



cv2.undistortPoints

Normalization

Normalized Coordinates on Z=1 Plane

$$\begin{bmatrix} x_{\text{norm}} \\ y_{\text{norm}} \end{bmatrix} = \text{undistort}(u, v, \mathbf{K}, \text{dist})$$

Project

Ray Construction

Unit Direction Vector

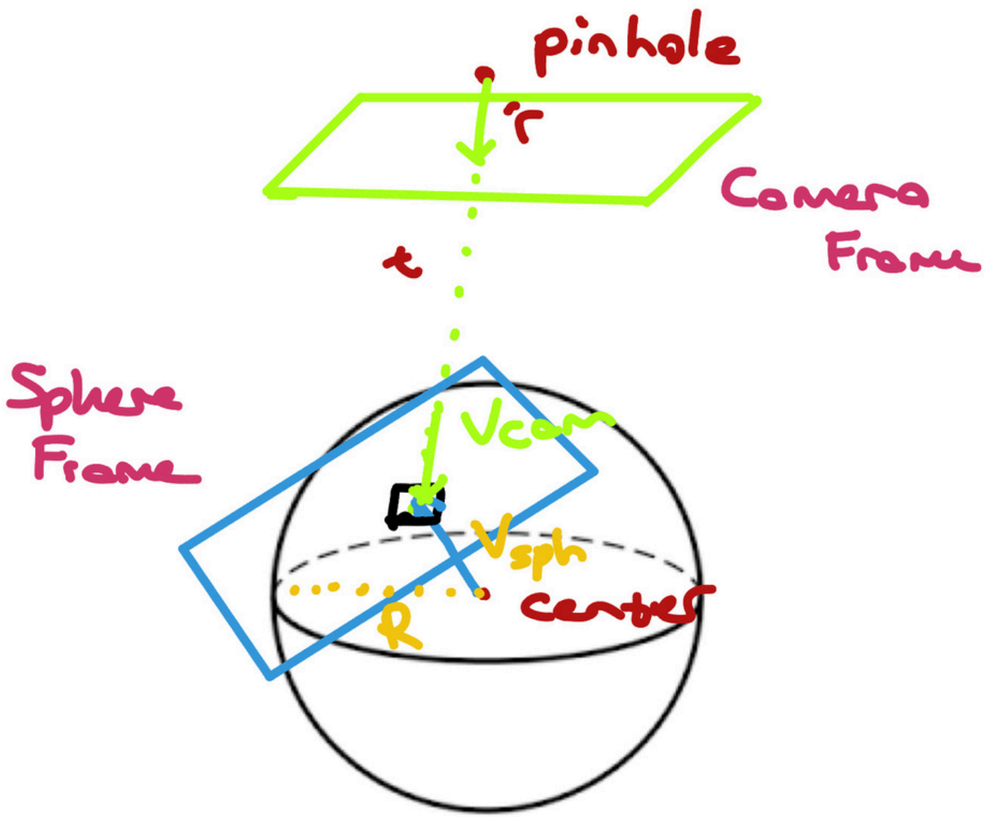
$$\mathbf{r} = \begin{bmatrix} x_{\text{norm}} \\ y_{\text{norm}} \\ 1.0 \end{bmatrix}$$

$$\hat{\mathbf{r}} = \frac{\mathbf{r}}{\|\mathbf{r}\|}$$

Result: A directional ray pointing from camera origin to marker. Distance is still unknown.

Step 3: Ray-Sphere Intersection Solver

The Geometric Core.



Quadratic Solver

Sphere Equation: $\|P - C\|^2 = R^2$

Ray Equation: $P = t * \hat{r}$

Substitute): $\|t * \hat{r} - C\|^2 = R^2$

Expand to Quadratic:

$$t^2 - 2t(\hat{r} \cdot C) + ||C||^2 - R^2 = 0$$

Solve for t using quadratic formula.
Select smaller positive t.

Intersection Point $P_{cam} = t * \hat{r}$

Step 4: Observed Unit Vector Derivation

$$\mathbf{v}_{cam} = \frac{P_{cam} - C_{cam}}{R}$$

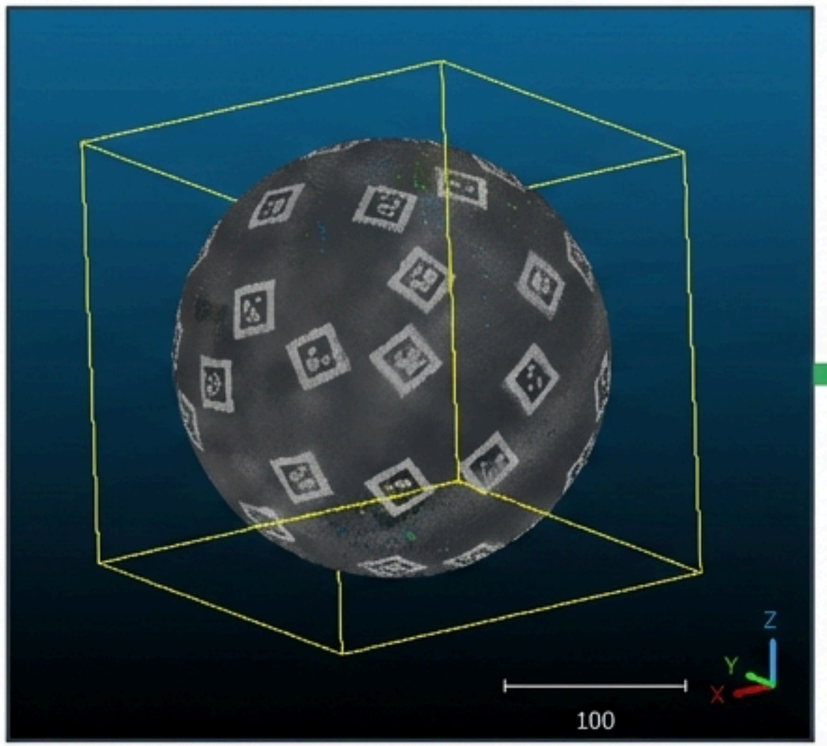
Converts the 3D surface point into a bearing vector originating from the sphere center.

Traditional Method (Spherical Coords)	Our Method (Vector Algebra)
Uses Latitude (φ) / Longitude (λ)	Uses Unit Vectors (x, y, z)
Requires: sin(), cos(), atan2()	Requires: Subtraction & Division
Heavy computational load per frame.	Result: 30x Computational Speedup.

Step 5: The Correspondence Problem

Matching Real-time Observation with Ground Truth.

Known Truth (v_{sphere})

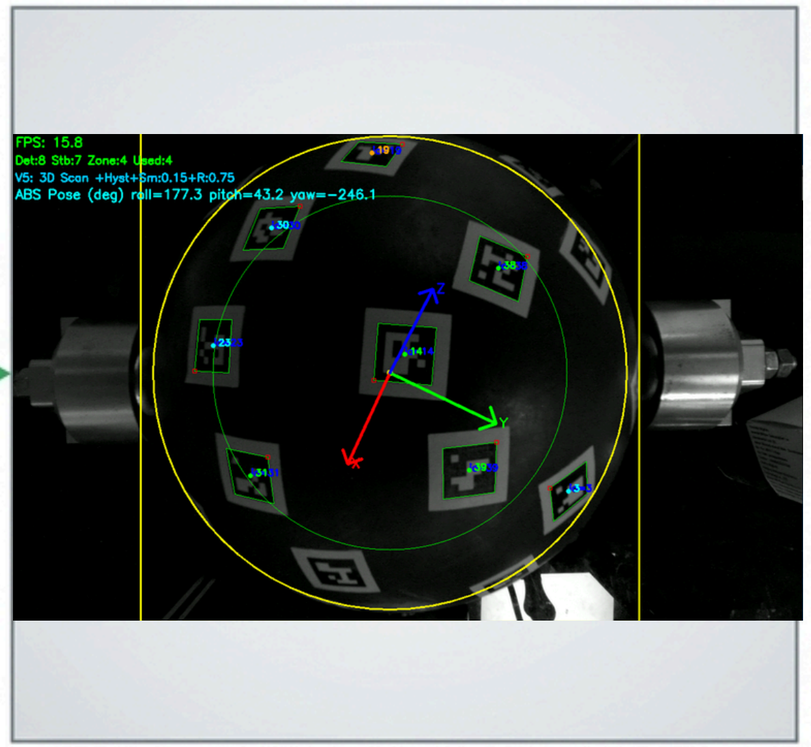


Source: High-resolution Structured Light Scan.
Data: Fixed vectors defined in local sphere frame.

$$v_{cam}(i) = R * v_{sphere}(i)$$

The Unknown Rotation Matrix we must find.

Observed Data (v_{cam})



Source: Real-time Camera Pipeline.
Data: Vectors defined in camera frame.

Step 6: Attitude Estimation (Wahba's Problem)

Finding the optimal rotation in the presence of noise.

Because of pixel noise, **no single rotation matrix R** fits all marker pairs perfectly. We must **minimize the error.**

$$J(\mathbf{R}) = \sum_{i=1}^{N} \| \mathbf{R} \cdot \mathbf{v}_{sph}(i) - \mathbf{v}_{cam}(i) \|^2$$

Constraints:

- $N \geq 3$ markers visible (to constrain 6 DoF).
- $R^T R = I$ (Orthogonal).
- $\det(R) = +1$ (No reflections).

Step 7: The Kabsch Algorithm (SVD Solution)

The mathematical engine of the tracker.

Cross-Covariance Matrix

$$\mathbf{H} = \sum_{i=1}^{N} \mathbf{v}_{\text{sph}}(i)\mathbf{v}_{\text{cam}}(i)^{\mathrm{T}}$$

Sum of outer products between corresponding vectors.

↓

Singular Value Decomposition (The Core)

$$H = U\ \Sigma\ V^{T}$$

Decomposes H into orthogonal matrices U, V and diagonal matrix Σ.

↓

Optimal Rotation

$$R = V\ U^{T}$$ ← Removes scaling/shear, extracting pure rotation.

↓

Chirality Check

If det(R) < 0 (Reflection detected), negate the last column of V.

Algorithm Implementation (Python)

Running on Raspberry Pi 5 @ 30Hz.

```python
def kabsch_rotation(V_from, V_to):
    # 1. Compute Cross-Covariance Matrix
    H = V_from.T @ V_to

    # 2. Singular Value Decomposition (SVD)
    U, S, Vt = np.linalg.svd(H)

    # 3. Compute Rotation Matrix
    R = Vt.T @ U.T

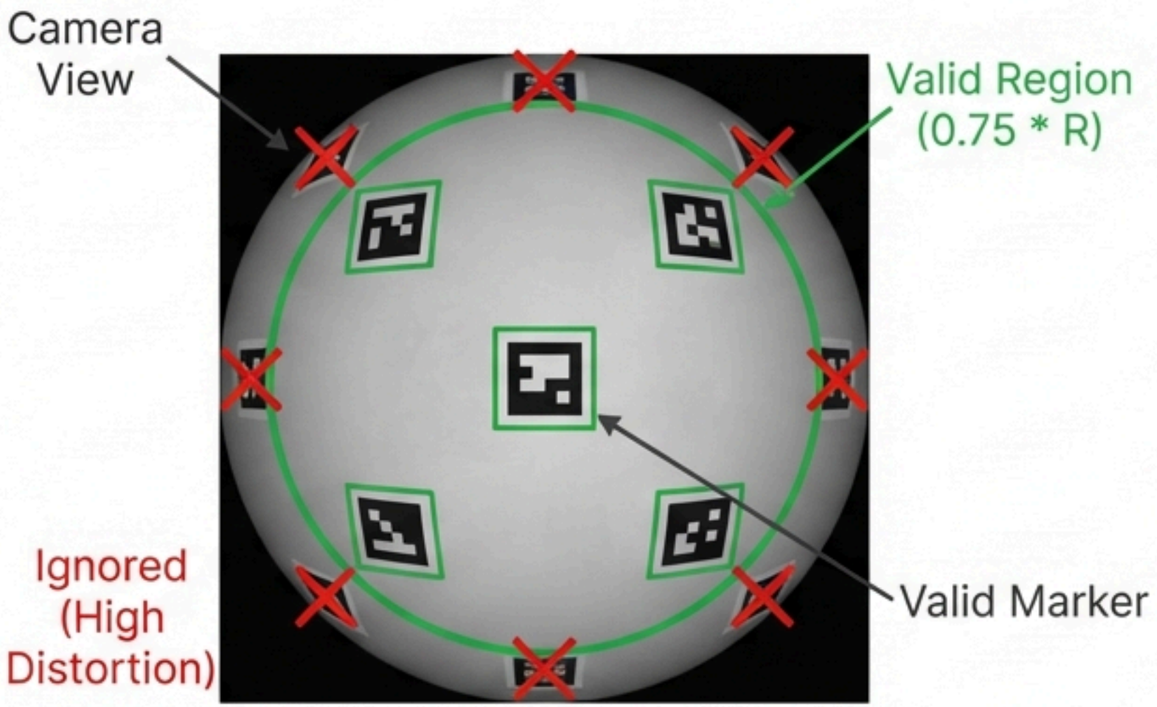
    # 4. Correction for Reflection Case
    if np.linalg.det(R) < 0:
        Vt[-1, :] *= -1
        R = Vt.T @ U.T

    return R
```

Utilizes `numpy.linalg.svd` for highly optimized linear algebra operations.

Robustness & Filtering Techniques

Spatial Gating (Radius Limit)



Temporal Hysteresis State Machine



Prevents detection flicker at the sphere edges.

Defining & Quantifying Drift

Mathematically separating **Truth** from **Estimate**.

Encoder Stream: $(\phi_{enc}, \theta_{enc}, \psi_{enc})$

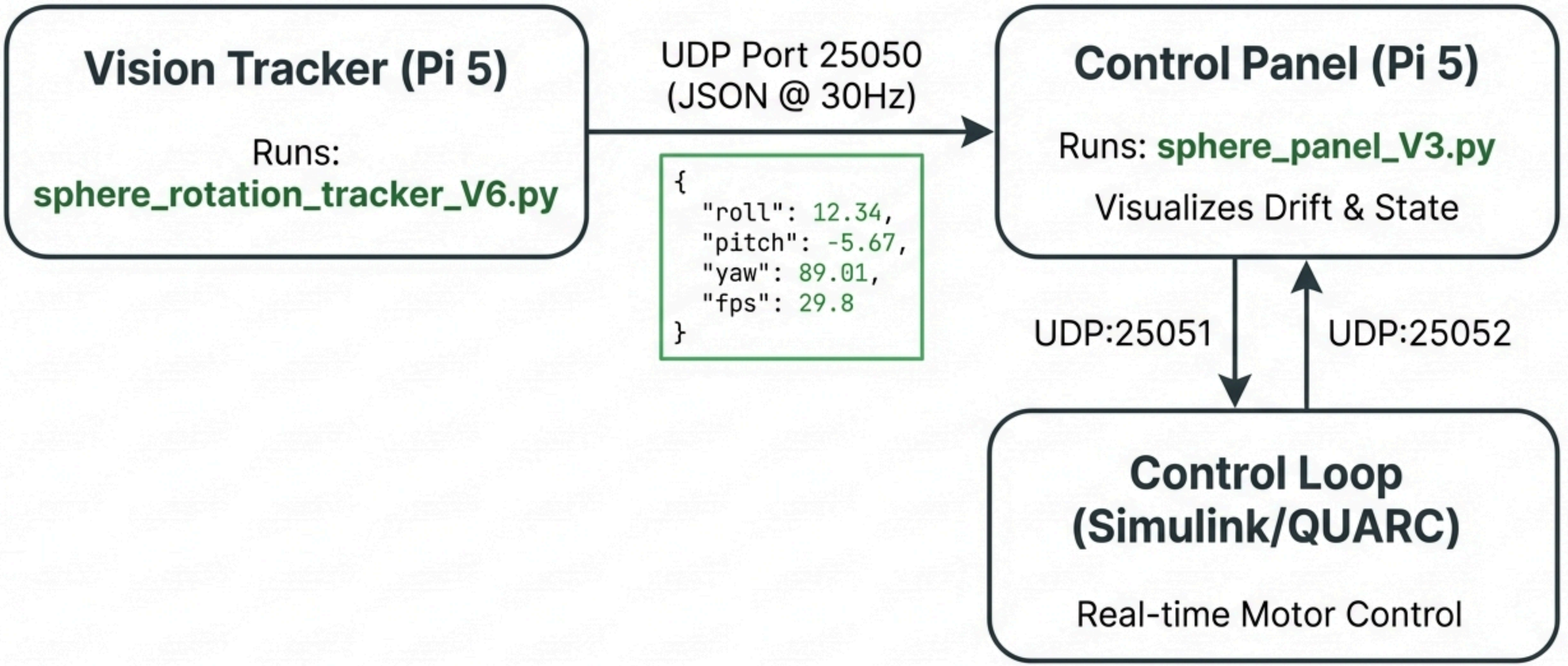
$$\text{Drift}(t) = \sqrt{(\phi_{enc} - \phi_{cam})^2 + (\theta_{enc} - \theta_{cam})^2 + (\psi_{enc} - \psi_{cam})^2}$$

Vision Stream: $(\phi_{cam}, \theta_{cam}, \psi_{cam})$

OK: < 6° → WARNING: 6° - 10° → ALARM: > 10°

Alarm state indicates significant traction loss.

System Integration Architecture



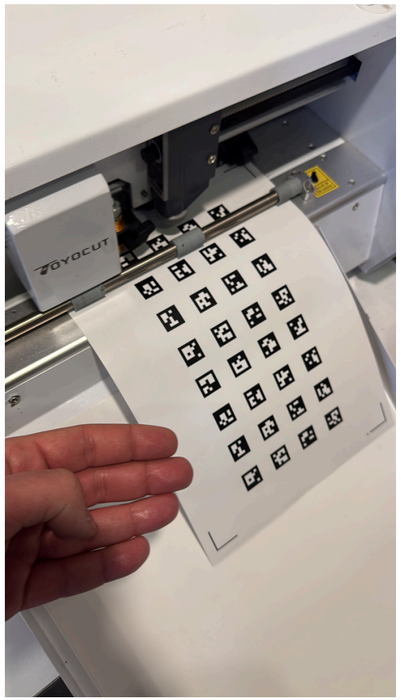
Technical Impact & Conclusion

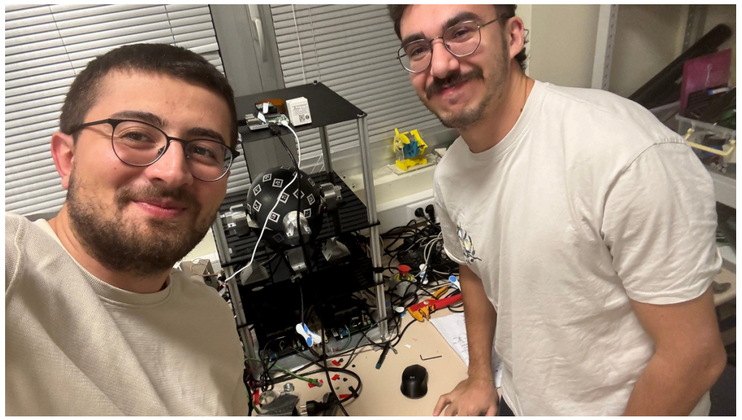
Key Achievements

1. **Absolute Tracking:** Sub-degree accuracy using bearing-only geometry.
2. **Observability:** Transformed 'Invisible Slip' into a quantifiable real-time metric.
3. **Efficiency:** 30x speedup by replacing trigonometry with linear algebra (SVD).



"By combining rigorous geometry with efficient computer vision, we bridged the gap between theoretical kinematics and physical reality."







Thank You !

